## Data Structures Home Work Set 7

### Trees and Graphs

Register No:2025115058

Name:Sri Senthil Adavan K

### Q1. Write a C program to construct an expression tree for a given expression. Example: (a-b)+(c*d).

*Code:*

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
struct Node { char data; struct Node *left, *right; };
struct Node *make(char c) { struct Node *p=malloc(sizeof(*p)); p->data=c; p->left=p->right=NULL; return p; }
int priority(char c) { return (c=='+'||c=='-')?1:(c=='*'||c=='/')?2:0; }
int main(void) {
   char infix[100], post[100], ops[100]; int i,k=0,top=-1;
   struct Node *tree[100], *p; int t=-1;
   printf("Enter infix expression: "); scanf("%99s",infix);
   for(i=0;infix[i];i++) {
      char c=infix[i];
      if(isalnum((unsigned char)c)) post[k++]=c;
      else if(c=='(') ops[++top]=c;
      else if(c==')') { while(top>=0&&ops[top]!='(') post[k++]=ops[top--]; if(top>=0)top--; }
      else { while(top>=0&&ops[top]!='('&&priority(ops[top])>=priority(c)) post[k++]=ops[top--]; ops[++top]=c; }
   }
   while(top>=0) post[k++]=ops[top--]; post[k]='\0';
   for(i=0;post[i];i++) {
      p=make(post[i]);
      if(!isalnum((unsigned char)post[i])) { p->right=tree[t--]; p->left=tree[t--]; }
      tree[++t]=p;
   }
   printf("Postfix form: %s\n",post);
   printf("Expression tree constructed successfully. Root operator: %c\n",tree[t]->data);
   return 0;
}
```

*Output:*

```
Enter infix expression: (a-b)+(c*d)
Postfix form: ab-cd*+
Expression tree constructed successfully. Root operator: +
```

### Q2. Write a C program to check whether a given binary tree is a binary search tree.

*Code:*

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
struct Node { int data; struct Node *left,*right; };
struct Node *node(int x) { struct Node *p=malloc(sizeof(*p)); p->data=x;p->left=p->right=NULL;return p; }
int isBST(struct Node *r,long long low,long long high) {
   if(!r) return 1;
```

```
    if(r->data<=low||r->data>=high) return 0;
    return isBST(r->left,low,r->data)&&isBST(r->right,r->data,high);
}
int main(void) {
    struct Node *root=node(40);
    root->left=node(20);root->right=node(60);
    root->left->left=node(10);root->left->right=node(30);
    if(isBST(root,LLONG_MIN,LLONG_MAX)) printf("The binary tree is a BST\n");
    else printf("The binary tree is not a BST\n");
    return 0;
}
```

*Output:*

```
The binary tree is a BST
```

### Q3. Write a C program to implement a max heap.

*Code:*

```
#include <stdio.h>
#define SIZE 100
int heap[SIZE], n=0;
void insert(int x) {
    int i=n++; heap[i]=x;
    while(i>0&&heap[(i-1)/2]<heap[i]) { int t=heap[i];heap[i]=heap[(i-1)/2];heap[(i-1)/2]=t;i=(i-1)/2; }
}
int removeMax(void) {
    int result=heap[0],i=0; heap[0]=heap[--n];
    while(2*i+1<n) { int child=2*i+1;if(child+1<n&&heap[child+1]>heap[child])child++;
        if(heap[i]>=heap[child])break;int t=heap[i];heap[i]=heap[child];heap[child]=t;i=child; }
    return result;
}
int main(void) {
    int a[]={10,40,30,5,12},i;
    for(i=0;i<5;i++)insert(a[i]);
    printf("Max heap: ");for(i=0;i<n;i++)printf("%d ",heap[i]);
    printf("\nRemoved maximum: %d\n",removeMax());
    printf("Heap after removal: ");for(i=0;i<n;i++)printf("%d ",heap[i]);printf("\n");
    return 0;
}
```

*Output:*

```
Max heap: 40 12 30 5 10
Removed maximum: 40
Heap after removal: 30 12 10 5
```

### Q4. Given an undirected graph, check whether there is a cycle in the graph.

*Code:*

```
#include <stdio.h>
#define V 6
int graph[V][V],visited[V];
int hasCycle(int v,int parent) {
    int i;visited[v]=1;
```

```
    for(i=0;i<V;i++) if(graph[v][i]) {
        if(!visited[i]) { if(hasCycle(i,v))return 1; }
        else if(i!=parent)return 1;
    }
    return 0;
}
int main(void) {
    int i,cycle=0;
    graph[0][1]=graph[1][0]=1;graph[1][2]=graph[2][1]=1;
    graph[2][0]=graph[0][2]=1;graph[3][4]=graph[4][3]=1;
    for(i=0;i<V;i++)if(!visited[i]&&hasCycle(i,-1)){cycle=1;break;}
    if(cycle)printf("The graph contains a cycle\n");else printf("The graph does not contain a cycle\n");
    return 0;
}
```

*Output:*

```
The graph contains a cycle
```

### Q5. Given an N x M grid containing W (water) and L (land), count the number of islands.

*This solution treats land cells as connected in all eight directions, including diagonals.*

*Code:*

```
#include <stdio.h>
#define N 4
#define M 5
char grid[N][M]={{'L','L','W','W','W'},{'W','L','W','W','L'},{'L','W','W','L','L'},{'W','W','W','W','W'}};
int seen[N][M];
void visit(int r,int c) {
    int dr,dc,nr,nc;
    if(r<0||r>=N||c<0||c>=M||seen[r][c]||grid[r][c]!='L')return;
    seen[r][c]=1;
    for(dr=-1;dr<=1;dr++)for(dc=-1;dc<=1;dc++)if(dr||dc){nr=r+dr;nc=c+dc;visit(nr,nc);}
}
int main(void) {
    int r,c,count=0;
    for(r=0;r<N;r++)for(c=0;c<M;c++)if(grid[r][c]=='L'&&!seen[r][c]){count++;visit(r,c);}
    printf("Number of islands = %d\n",count);return 0;
}
```

*Output:*

```
Number of islands = 2
```

### Q6. Find the minimum spanning tree using Kruskal's algorithm.

*Code:*

```
#include <stdio.h>
#define V 4
#define E 5
struct Edge { int u,v,w; };
int parent[V];
int find(int x){while(parent[x]!=x){parent[x]=parent[parent[x]];x=parent[x];}return x;}
int main(void) {
    struct Edge e[E]={{0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}},t;
```

```
    int i,j,used=0,total=0;
    for(i=0;i<V;i++)parent[i]=i;
    for(i=0;i<E-1;i++)for(j=0;j<E-i-1;j++)if(e[j].w>e[j+1].w){t=e[j];e[j]=e[j+1];e[j+1]=t;}
    printf("Edges in minimum spanning tree:\n");
    for(i=0;i<E&&used<V-1;i++){int a=find(e[i].u),b=find(e[i].v);if(a!=b){parent[a]=b;printf("%d -
%d : %d\n",e[i].u,e[i].v,e[i].w);total+=e[i].w;used++;}}
    printf("Total weight = %d\n",total);return 0;
}
```

*Output:*

```
Edges in minimum spanning tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total weight = 19
```

### Q7. Use Dijkstra's algorithm to find the shortest path from a source vertex to all other vertices.

*The adjacency matrix uses 0 to indicate that no edge exists. All edge weights must be non-negative.*

*Code:*

```
#include <stdio.h>
#define V 5
#define INF 99999
int main(void) {
    int g[V][V]={{0,10,0,5,0},{0,0,1,2,0},{0,0,0,0,4},{0,3,9,0,2},{7,0,6,0,0}};
    int d[V],used[V]={0},i,j,src=0,min,u;
    for(i=0;i<V;i++)d[i]=INF;d[src]=0;
    for(i=0;i<V;i++){min=INF;u=-1;for(j=0;j<V;j++)if(!used[j]&&d[j]<min){min=d[j];u=j;}if(u<0)break;used[u]=1;
        for(j=0;j<V;j++)if(g[u][j]>0&&!used[j]&&d[u]+g[u][j]<d[j])d[j]=d[u]+g[u][j];}
    printf("Shortest distances from vertex %d:\n",src);
    for(i=0;i<V;i++)if(d[i]==INF)printf("%d: unreachable\n",i);else printf("%d: %d\n",i,d[i]);
    return 0;
}
```

*Output:*

```
Shortest distances from vertex 0:
0: 0
1: 8
2: 9
3: 5
4: 7
```

### Q8. Write a program to find a mother vertex in a graph.

*A mother vertex can reach every vertex. This program uses a directed graph; if none exists, it reports that fact.*

*Code:*

```
#include <stdio.h>
#define V 5
int g[V][V],seen[V];
void dfs(int v){int i;seen[v]=1;for(i=0;i<V;i++)if(g[v][i]&&!seen[i])dfs(i);}
int main(void){
    int i,candidate=0,found=1;
```

```
    g[0][2]=1;g[0][3]=1;g[1][0]=1;g[2][1]=1;g[3][4]=1;
    for(i=0;i<V;i++)if(!seen[i]){dfs(i);candidate=i;}
    for(i=0;i<V;i++)seen[i]=0;dfs(candidate);
    for(i=0;i<V;i++)if(!seen[i])found=0;
    if(found)printf("Mother vertex = %d\n",candidate);else printf("No mother vertex exists\n");
    return 0;
}
```

*Output:*

```
Mother vertex = 0
```

### Q9. Given a directed graph, print all strongly connected components (SCCs).

*Code:*

```
#include <stdio.h>
#define V 5
int g[V][V],seen[V],stack[V],top=-1;
void fill(int v){int i;seen[v]=1;for(i=0;i<V;i++)if(g[v][i]&&!seen[i])fill(i);stack[++top]=v;}
void printDFS(int v,int tr[V][V]){int i;seen[v]=1;printf("%d ",v);for(i=0;i<V;i++)if(tr[v][i]&&!seen[i])printDFS(i,tr);}
int main(void){
    int tr[V][V]={{0}},i,j;
    g[0][1]=g[1][2]=g[2][0]=g[1][3]=g[3][4]=g[4][3]=1;
    for(i=0;i<V;i++)for(j=0;j<V;j++)tr[j][i]=g[i][j];
    for(i=0;i<V;i++)if(!seen[i])fill(i);
    for(i=0;i<V;i++)seen[i]=0;
    printf("Strongly connected components:\n");
    while(top>=0){int v=stack[top--];if(!seen[v]){printDFS(v,tr);printf("\n");}}
    return 0;
}
```

*Output:*

```
Strongly connected components:
0 2 1
3 4
```

### Q10. Write a program to determine whether a graph is bipartite.

*Code:*

```
#include <stdio.h>
#define V 5
int main(void){
    int g[V][V]={{0,1,0,0,1},{1,0,1,0,0},{0,1,0,1,0},{0,0,1,0,1},{1,0,0,1,0}};
    int color[V],q[V],front,rear,i,j,start,bipartite=1;
    for(i=0;i<V;i++)color[i]=-1;
    for(start=0;start<V;start++)if(color[start]==-1){front=rear=0;q[rear++]=start;color[start]=0;
        while(front<rear){int u=q[front++];for(j=0;j<V;j++)if(g[u][j]){if(color[j]==-1){color[j]=1-color[u];q[rear++]=j;}else
if(color[j]==color[u])bipartite=0;}}}
    if(bipartite)printf("The graph is bipartite\n");else printf("The graph is not bipartite\n");
    return 0;
}
```

*Output:*

```
The graph is not bipartite
```